

PART 4 REACT NATIVE COURSE



Part 4: Professional Development (Expert Level)

Chapter 13: Advanced State Management with Redux/Context

Understanding Enterprise-Level State Management

As React Native applications grow in complexity, managing state becomes increasingly challenging. Professional applications require sophisticated state management solutions that can handle complex data flows, asynchronous operations, and scalable architecture patterns.

When to Use Advanced State Management:

- **Multiple Screens:** State shared across many components
- **Complex Data:** Nested objects with frequent updates
- **User Sessions:** Authentication and user-specific data
- **Real-time Updates:** WebSocket connections and live data
- **Offline Support:** Caching and synchronization requirements

Redux vs Context API:

- **Redux:** Better for complex apps with predictable state updates
- **Context API:** Suitable for simpler state sharing needs
- **Redux Toolkit:** Modern, simplified Redux with less boilerplate
- **Zustand/Jotai:** Lightweight alternatives for specific use cases

```
// store/store.js - Redux Toolkit Configuration
import { configureStore } from '@reduxjs/toolkit';
import { persistStore, persistReducer } from 'redux-persist';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { combineReducers } from '@reduxjs/toolkit';

// Import reducers
import authReducer from './slices/authSlice';
import userReducer from './slices/userSlice';
import postsReducer from './slices/postsSlice';
import uiReducer from './slices/uiSlice';
import { apiSlice } from './api/apiSlice';

/**
 * Persist configuration for Redux
 * Automatically saves and restores state from AsyncStorage
 */
const persistConfig = {
  key: 'root',
  storage: AsyncStorage,
  // Only persist certain reducers
```

 Copy

```

whitelist: ['auth', 'user'],
// Blacklist sensitive or temporary data
blacklist: ['ui'],
// Version for migration support
version: 1,
// Custom transformation functions
transforms: [
  // Transform sensitive data before storing
  {
    in: (state, key) => {
      if (key === 'auth' && state.token) {
        // Encrypt sensitive data before storage
        return {
          ...state,
          token: encryptToken(state.token),
        };
      }
      return state;
    },
    out: (state, key) => {
      if (key === 'auth' && state.token) {
        // Decrypt sensitive data after loading
        return {
          ...state,
          token: decryptToken(state.token),
        };
      }
      return state;
    },
  },
],
};

// Combine all reducers
const rootReducer = combineReducers({
  auth: authReducer,
  user: userReducer,
  posts: postsReducer,
  ui: uiReducer,
  // RTK Query API reducer
  api: apiSlice.reducer,
});

// Create persisted reducer
const persistedReducer = persistReducer(persistConfig, rootReducer);

/**
 * Configure Redux store with middleware
 * Includes development tools and persistence

```

```

*/
export const store = configureStore({
  reducer: persistedReducer,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      // Configure serializable check for redux-persist
      serializableCheck: {
        ignoredActions: ['persist/PERSIST', 'persist/REHYDRATE'],
        ignoredActionsPaths: ['register', 'rehydrate'],
        ignoredPaths: ['_persist'],
      },
      // Disable immutability check in production for performance
      immutableCheck: __DEV__,
    }).concat(
      // Add RTK Query middleware
      apiSlice.middleware,
      // Custom middleware for logging
      __DEV__ ? logger : [],
    ),
  // Enable Redux DevTools in development
  devTools: __DEV__,
  // Preloaded state for testing or SSR
  preloadedState: undefined,
});

// Create persistor for state persistence
export const persistor = persistStore(store);

// Export types for TypeScript support
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

/**
 * Custom logger middleware for debugging
 */
const logger = (store) => (next) => (action) => {
  console.group(`🔵 Action: ${action.type}`);
  console.log('📄 Payload:', action.payload);
  console.log('📄 Previous State:', store.getState());

  const result = next(action);

  console.log('📄 Next State:', store.getState());
  console.groupEnd();

  return result;
};

/**

```

```

* Token encryption utilities
* Implement proper encryption in production
*/
const encryptToken = (token) => {
  // Implement actual encryption here
  return btoa(token); // Base64 encoding (not secure, use proper encryption)
};

const decryptToken = (encryptedToken) => {
  try {
    return atob(encryptedToken);
  } catch (error) {
    console.error('Token decryption failed:', error);
    return null;
  }
};

```

Redux Slices with RTK Query

 Copy

```

// store/slices/authSlice.js - Authentication State Management
import { createSlice, createAsyncThunk, PayloadAction } from '@reduxjs/toolkit';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { authAPI } from '../api/authAPI';

/**
 * Authentication state interface
 */
interface AuthState {
  user: User | null;
  token: string | null;
  refreshToken: string | null;
  isLoading: boolean;
  isAuthenticated: boolean;
  error: string | null;
  loginAttempts: number;
  lastLoginAttempt: number | null;
  biometricsEnabled: boolean;
  sessionExpiry: number | null;
}

/**
 * Initial authentication state
 */
const initialState: AuthState = {
  user: null,
  token: null,
  refreshToken: null,

```

```
    isLoading: false,
    isAuthenticated: false,
    error: null,
    loginAttempts: 0,
    lastLoginAttempt: null,
    biometricsEnabled: false,
    sessionExpiry: null,
  };

/**
 * Async thunk for user login
 * Handles complex async logic with proper error handling
 */
export const loginUser = createAsyncThunk(
  'auth/loginUser',
  async (credentials: LoginCredentials, { rejectWithValue, dispatch }) => {
    try {
      // Call API with credentials
      const response = await authAPI.login(credentials);

      // Store tokens securely
      await AsyncStorage.setItem('@auth_token', response.token);
      await AsyncStorage.setItem('@refresh_token', response.refreshToken);

      // Set session expiry
      const expiryTime = Date.now() + (response.expiresIn * 1000);
      await AsyncStorage.setItem('@session_expiry', expiryTime.toString());

      // Log successful login for analytics
      dispatch(logEvent('user_login_success'));

      return {
        user: response.user,
        token: response.token,
        refreshToken: response.refreshToken,
        sessionExpiry: expiryTime,
      };
    } catch (error) {
      // Handle different error types
      if (error.status === 401) {
        return rejectWithValue('Invalid credentials');
      } else if (error.status === 429) {
        return rejectWithValue('Too many login attempts. Please try again later.');
```

```
      } else if (error.status >= 500) {
        return rejectWithValue('Server error. Please try again.');
```

```
      }

      return rejectWithValue(error.message || 'Login failed');
```

```

    }
  }
};

/**
 * Async thunk for token refresh
 */
export const refreshAuthToken = createAsyncThunk(
  'auth/refreshToken',
  async (_, { getState, rejectWithValue }) => {
    try {
      const { auth } = getState() as { auth: AuthState };

      if (!auth.refreshToken) {
        throw new Error('No refresh token available');
      }

      const response = await authAPI.refreshToken(auth.refreshToken);

      // Update stored tokens
      await AsyncStorage.setItem('@auth_token', response.token);
      const expiryTime = Date.now() + (response.expiresIn * 1000);
      await AsyncStorage.setItem('@session_expiry', expiryTime.toString());

      return {
        token: response.token,
        sessionExpiry: expiryTime,
      };
    } catch (error) {
      // Clear invalid tokens
      await AsyncStorage.multiRemove(['@auth_token', '@refresh_token',
        '@session_expiry']);
      return rejectWithValue('Token refresh failed');
    }
  }
);

/**
 * Async thunk for logout
 */
export const logoutUser = createAsyncThunk(
  'auth/logoutUser',
  async (_, { getState, dispatch }) => {
    try {
      const { auth } = getState() as { auth: AuthState };

      // Call logout API if token exists
      if (auth.token) {
        await authAPI.logout(auth.token);
      }
    }
  }
);

```

```

    }

    // Clear all stored auth data
    await AsyncStorage.multiRemove([
      '@auth_token',
      '@refresh_token',
      '@session_expiry',
      '@user_profile',
    ]);

    // Clear other user-specific data
    dispatch(clearUserData());

    // Log logout event
    dispatch(logEvent('user_logout'));

  } catch (error) {
    // Even if API call fails, clear local data
    console.error('Logout API call failed:', error);
  }
}

/**
 * Async thunk for biometric authentication
 */
export const authenticateWithBiometrics = createAsyncThunk(
  'auth/biometricAuth',
  async (_, { rejectWithValue }) => {
    try {
      // Check if biometrics are available
      const isBiometricSupported = await BiometricAuth.isSupported();

      if (!isBiometricSupported) {
        return rejectWithValue('Biometric authentication not supported');
      }

      // Prompt for biometric authentication
      const result = await BiometricAuth.authenticate({
        promptMessage: 'Authenticate to access your account',
        fallbackPromptMessage: 'Use device passcode',
        cancelButtonText: 'Cancel',
      });

      if (result.success) {
        // Retrieve stored credentials
        const storedToken = await AsyncStorage.getItem('@auth_token');
        const storedUser = await AsyncStorage.getItem('@user_profile');

```

```

    if (storedToken && storedUser) {
      return {
        token: storedToken,
        user: JSON.parse(storedUser),
      };
    }

    return rejectWithValue('Biometric authentication failed');
  } catch (error) {
    return rejectWithValue(error.message);
  }
}
);

/**
 * Auth slice with reducers
 */
const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    // Synchronous actions
    clearError: (state) => {
      state.error = null;
    },

    resetLoginAttempts: (state) => {
      state.loginAttempts = 0;
      state.lastLoginAttempt = null;
    },

    setBiometricsEnabled: (state, action: PayloadAction<boolean>) => {
      state.biometricsEnabled = action.payload;
    },

    updateUserProfile: (state, action: PayloadAction<Partial<User>>) => {
      if (state.user) {
        state.user = { ...state.user, ...action.payload };
      }
    },

    setSessionExpiry: (state, action: PayloadAction<number>) => {
      state.sessionExpiry = action.payload;
    },
  },
  extraReducers: (builder) => {
    // Login user cases
    builder

```

```
.addCase(loginUser.pending, (state) => {
  state.isLoading = true;
  state.error = null;
})

.addCase(loginUser.fulfilled, (state, action) => {
  state.isLoading = false;
  state.isAuthenticated = true;
  state.user = action.payload.user;
  state.token = action.payload.token;
  state.refreshToken = action.payload.refreshToken;
  state.sessionExpiry = action.payload.sessionExpiry;
  state.loginAttempts = 0;
  state.error = null;
})

.addCase(loginUser.rejected, (state, action) => {
  state.isLoading = false;
  state.error = action.payload as string;
  state.loginAttempts += 1;
  state.lastLoginAttempt = Date.now();
});

// Token refresh cases
builder
.addCase(refreshAuthToken.fulfilled, (state, action) => {
  state.token = action.payload.token;
  state.sessionExpiry = action.payload.sessionExpiry;
})
.addCase(refreshAuthToken.rejected, (state) => {
  // Force logout on refresh failure
  state.user = null;
  state.token = null;
  state.refreshToken = null;
  state.isAuthenticated = false;
  state.sessionExpiry = null;
});

// Logout cases
builder.addCase(logoutUser.fulfilled, (state) => {
  return { ...initialState }; // Reset to initial state
});

// Biometric auth cases
builder
.addCase(authenticateWithBiometrics.pending, (state) => {
  state.isLoading = true;
})
.addCase(authenticateWithBiometrics.fulfilled, (state, action) => {
  state.isLoading = false;
  state.isAuthenticated = true;
```

```

        state.user = action.payload.user;
        state.token = action.payload.token;
    })
    .addCase(authenticateWithBiometrics.rejected, (state, action) => {
        state.isLoading = false;
        state.error = action.payload as string;
    });
},
});

// Export actions
export const {
    clearError,
    resetLoginAttempts,
    setBiometricsEnabled,
    updateUserProfile,
    setSessionExpiry,
} = authSlice.actions;

// Export selectors
export const selectAuth = (state: { auth: AuthState }) => state.auth;
export const selectIsAuthenticated = (state: { auth: AuthState }) =>
state.auth.isAuthenticated;
export const selectUser = (state: { auth: AuthState }) => state.auth.user;
export const selectAuthError = (state: { auth: AuthState }) => state.auth.error;
export const selectIsLoading = (state: { auth: AuthState }) =>
state.auth.isLoading;

// Export reducer
export default authSlice.reducer;

```

RTK Query API Integration

```

// store/api/apiSlice.js - Centralized API Management
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';
import AsyncStorage from '@react-native-async-storage/async-storage';

/**
 * Base query with automatic token handling
 */
const baseQuery = fetchBaseQuery({
    baseUrl: 'https://api.yourapp.com/v1',
    prepareHeaders: async (headers, { getState }) => {
        // Get token from state or AsyncStorage
        const token = getState()?.auth?.token || await
        AsyncStorage.getItem('@auth_token');
    }
});

```

 Copy

```
    if (token) {
      headers.set('authorization', `Bearer ${token}`);
    }

    // Set content type
    headers.set('content-type', 'application/json');

    // Add client info for analytics
    headers.set('x-client-platform', 'react-native');
    headers.set('x-client-version', '1.0.0');

    return headers;
  },
});

/**
 * Base query with automatic token refresh
 */
const baseQueryWithReauth = async (args, api, extraOptions) => {
  let result = await baseQuery(args, api, extraOptions);

  // Handle token expiration
  if (result?.error?.status === 401) {
    console.log('Token expired, attempting refresh...');

    // Try to refresh the token
    const refreshResult = await api.dispatch(refreshAuthToken());

    if (refreshResult.meta.requestStatus === 'fulfilled') {
      // Retry the original query with new token
      result = await baseQuery(args, api, extraOptions);
    } else {
      // Refresh failed, logout user
      api.dispatch(logoutUser());
    }
  }

  return result;
};

/**
 * Main API slice with all endpoints
 */
export const apiSlice = createApi({
  reducerPath: 'api',
  baseQuery: baseQueryWithReauth,
  // Tags for cache invalidation
  tagTypes: ['User', 'Post', 'Comment', 'Notification'],
  // Global error handling
```

```

endpoints: (builder) => ({
  // User endpoints
  getUserProfile: builder.query({
    query: (userId) => `/users/${userId}`,
    providesTags: ['User'],
    // Transform response data
    transformResponse: (response) => ({
      ...response.data,
      fullName: `${response.data.firstName} ${response.data.lastName}`,
    }),
  }),

  updateUserProfile: builder.mutation({
    query: ({ userId, data }) => ({
      url: `/users/${userId}`,
      method: 'PUT',
      body: data,
    }),
    invalidatesTags: ['User'],
    // Optimistic update
    onQueryStarted: async ({ userId, data }, { dispatch, queryFulfilled }) => {
      const patchResult = dispatch(
        apiSlice.util.updateQueryData('getUserProfile', userId, (draft) => {
          Object.assign(draft, data);
        })
      );

      try {
        await queryFulfilled;
      } catch {
        // Revert optimistic update on failure
        patchResult.undo();
      }
    },
  }),

  // Posts endpoints
  getPosts: builder.query({
    query: ({ page = 1, limit = 10, category } = {}) => ({
      url: '/posts',
      params: { page, limit, category },
    }),
    providesTags: (result) =>
      result
        ? [
            ...result.data.map(({ id }) => ({ type: 'Post', id })),
            { type: 'Post', id: 'LIST' },
          ]
        : [{ type: 'Post', id: 'LIST' }],
  }),
});

```

```

// Merge pages for infinite scroll
serializeQueryArgs: ({ endpointName, queryArgs }) => {
  const { category } = queryArgs;
  return `${endpointName}(${category || 'all'})`;
},
merge: (currentCache, newItems, { arg }) => {
  if (arg.page === 1) {
    // Reset cache for first page
    currentCache.data = newItems.data;
  } else {
    // Append new items for subsequent pages
    currentCache.data.push(...newItems.data);
  }
  currentCache.pagination = newItems.pagination;
},
forceRefetch({ currentArg, previousArg }) {
  return currentArg?.page !== previousArg?.page;
},
}),

createPost: builder.mutation({
  query: (postData) => ({
    url: '/posts',
    method: 'POST',
    body: postData,
  }),
  invalidatesTags: [{ type: 'Post', id: 'LIST' }],
}),

uploadImage: builder.mutation({
  query: ({ imageUri, filename }) => {
    const formData = new FormData();
    formData.append('image', {
      uri: imageUri,
      type: 'image/jpeg',
      name: filename,
    } as any);

    return {
      url: '/upload/image',
      method: 'POST',
      body: formData,
      headers: {
        'content-type': 'multipart/form-data',
      },
    };
  },
}),

```

```

// Real-time notifications
getNotifications: builder.query({
  query: () => '/notifications',
  providesTags: ['Notification'],
  // Poll for new notifications
  pollingInterval: 30000, // 30 seconds
}),

markNotificationAsRead: builder.mutation({
  query: (notificationId) => ({
    url: `/notifications/${notificationId}/read`,
    method: 'PUT',
  }),
  invalidatesTags: ['Notification'],
  // Optimistic update
  onQueryStarted: async (notificationId, { dispatch, queryFulfilled }) => {
    const patchResult = dispatch(
      apiSlice.util.updateQueryData('getNotifications', undefined, (draft) =>
{
      const notification = draft.find((n) => n.id === notificationId);
      if (notification) {
        notification.isRead = true;
      }
    })
  );

    try {
      await queryFulfilled;
    } catch {
      patchResult.undo();
    }
  },
  }),
  });

// Export hooks for components
export const {
  useGetUserProfileQuery,
  useUpdateUserProfileMutation,
  useGetPostsQuery,
  useCreatePostMutation,
  useUploadImageMutation,
  useGetNotificationsQuery,
  useMarkNotificationAsReadMutation,
} = apiSlice;

// Export util functions for manual cache management

```

```
export const { updateQueryData, invalidateTags } = apiSlice.util;
```

Chapter 14: Authentication and Security

Comprehensive Authentication System

Security is paramount in mobile applications, especially when handling user data and sensitive information. A robust authentication system must handle various scenarios including biometric authentication, token management, and secure storage.

Security Considerations:

- **Token Storage:** Secure storage of authentication tokens
- **Biometric Integration:** Fingerprint and Face ID authentication
- **Session Management:** Automatic token refresh and expiration
- **Data Encryption:** Protecting sensitive data at rest and in transit
- **Certificate Pinning:** Preventing man-in-the-middle attacks

```
// services/AuthService.js - Comprehensive Authentication Service
import AsyncStorage from '@react-native-async-storage/async-storage';
import * as Keychain from 'react-native-keychain';
import * as LocalAuthentication from 'expo-local-authentication';
import CryptoJS from 'crypto-js';
import DeviceInfo from 'react-native-device-info';

/**
 * Authentication Service Class
 * Handles all authentication-related operations
 */
class AuthService {
  private static instance: AuthService;
  private readonly ENCRYPTION_KEY = 'your-secret-key';
  private readonly TOKEN_KEY = 'auth_token';
  private readonly REFRESH_TOKEN_KEY = 'refresh_token';
  private readonly USER_KEY = 'user_profile';

  // Session management
  private tokenRefreshTimer?: NodeJS.Timeout;
  private sessionWarningTimer?: NodeJS.Timeout;

  /**
   * Singleton pattern for service instance
   */
  public static getInstance(): AuthService {
    if (!AuthService.instance) {
```

 Copy

```
    AuthService.instance = new AuthService();
  }
  return AuthService.instance;
}

/**
 * Initialize authentication service
 * Set up automatic token refresh and session monitoring
 */
public async initialize(): Promise<void> {
  try {
    // Check for existing session
    const token = await this.getStoredToken();

    if (token) {
      // Verify token validity
      const isValid = await this.verifyToken(token);

      if (isValid) {
        // Set up automatic refresh
        await this.setupTokenRefresh();

        // Set up session monitoring
        this.setupSessionMonitoring();
      } else {
        // Clear invalid token
        await this.clearAuthData();
      }
    }

    // Set up device security checks
    await this.checkDeviceSecurity();

  } catch (error) {
    console.error('Auth service initialization failed:', error);
  }
}

/**
 * Login with email and password
 */
public async login(credentials: LoginCredentials): Promise<AuthResult> {
  try {
    // Add device information to login request
    const deviceInfo = await this.getDeviceInfo();

    const response = await fetch('/api/auth/login', {
      method: 'POST',
      headers: {
```

```

        'Content-Type': 'application/json',
        'X-Device-ID': deviceInfo.deviceId,
        'X-Device-Platform': deviceInfo.platform,
    },
    body: JSON.stringify({
        ...credentials,
        deviceInfo,
    }),
});

if (!response.ok) {
    const errorData = await response.json();
    throw new Error(errorData.message || 'Login failed');
}

const authData = await response.json();

// Store authentication data securely
await this.storeAuthData(authData);

// Set up session management
await this.setupTokenRefresh();
this.setupSessionMonitoring();

return {
    success: true,
    user: authData.user,
    token: authData.token,
};

} catch (error) {
    return {
        success: false,
        error: error.message,
    };
}
}

/**
 * Biometric authentication
 */
public async loginWithBiometrics(): Promise<AuthResult> {
    try {
        // Check biometric availability
        const biometricType = await
LocalAuthentication.supportedAuthenticationTypesAsync();

        if (biometricType.length === 0) {
            throw new Error('Biometric authentication not available');
        }
    }
}

```

```
}

// Prompt for biometric authentication
const biometricResult = await LocalAuthentication.authenticateAsync({
  promptMessage: 'Authenticate to access your account',
  fallbackLabel: 'Use Passcode',
  cancelLabel: 'Cancel',
  requireConfirmation: true,
});

if (!biometricResult.success) {
  throw new Error('Biometric authentication failed');
}

// Retrieve stored credentials from Keychain
const credentials = await Keychain.getInternetCredentials('app_auth');

if (!credentials || credentials.password === false) {
  throw new Error('No stored credentials found');
}

// Decrypt stored token
const decryptedToken = this.decrypt(credentials.password);

// Verify token is still valid
const isValid = await this.verifyToken(decryptedToken);

if (!isValid) {
  // Clear invalid credentials
  await Keychain.resetInternetCredentials('app_auth');
  throw new Error('Stored session expired');
}

// Retrieve user data
const userData = await AsyncStorage.getItem(this.USER_KEY);

if (!userData) {
  throw new Error('User data not found');
}

return {
  success: true,
  user: JSON.parse(userData),
  token: decryptedToken,
};

} catch (error) {
  return {
    success: false,
```

```

        error: error.message,
    };
}
}

/**
 * Enable biometric authentication
 */
public async enableBiometrics(token: string): Promise<boolean> {
    try {
        // Check if biometrics are supported
        const hasHardware = await LocalAuthentication.hasHardwareAsync();
        const isEnrolled = await LocalAuthentication.isEnrolledAsync();

        if (!hasHardware || !isEnrolled) {
            throw new Error('Biometric authentication not available');
        }

        // Encrypt and store token in Keychain
        const encryptedToken = this.encrypt(token);

        await Keychain.setInternetCredentials(
            'app_auth',
            'user_token',
            encryptedToken,
            {
                accessControl: Keychain.ACCESS_CONTROL.BIOMETRY_ANY,
                authenticationType: Keychain.AUTHENTICATION_TYPE.BIOMETRICS,
                accessGroup: 'your.app.keychain.group',
            }
        );

        return true;
    } catch (error) {
        console.error('Failed to enable biometrics:', error);
        return false;
    }
}

/**
 * Refresh authentication token
 */
public async refreshToken(): Promise<string | null> {
    try {
        const refreshToken = await AsyncStorage.getItem(this.REFRESH_TOKEN_KEY);

        if (!refreshToken) {
            throw new Error('No refresh token available');
        }
    }
}

```

```

const response = await fetch('/api/auth/refresh', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${refreshToken}`,
  },
});

if (!response.ok) {
  throw new Error('Token refresh failed');
}

const data = await response.json();

// Update stored tokens
await AsyncStorage.setItem(this.TOKEN_KEY, data.token);

if (data.refreshToken) {
  await AsyncStorage.setItem(this.REFRESH_TOKEN_KEY, data.refreshToken);
}

// Update Keychain if biometrics enabled
const biometricsEnabled = await this.isBiometricsEnabled();
if (biometricsEnabled) {
  const encryptedToken = this.encrypt(data.token);
  await Keychain.setInternetCredentials('app_auth', 'user_token',
encryptedToken);
}

return data.token;
} catch (error) {
  console.error('Token refresh failed:', error);
  // Clear invalid tokens
  await this.clearAuthData();
  return null;
}
}

/**
 * Logout user and clear all auth data
 */
public async logout(): Promise<void> {
  try {
    const token = await this.getStoredToken();

    // Call logout API
    if (token) {
      await fetch('/api/auth/logout', {

```

```

        method: 'POST',
        headers: {
            'Authorization': `Bearer ${token}`,
        },
    });
}
} catch (error) {
    console.error('Logout API call failed:', error);
} finally {
    // Always clear local data
    await this.clearAuthData();
    this.clearTimers();
}
}

/**
 * Store authentication data securely
 */
private async storeAuthData(authData: AuthData): Promise<void> {
    try {
        // Store in AsyncStorage
        await AsyncStorage.setItem(this.TOKEN_KEY, authData.token);
        await AsyncStorage.setItem(this.REFRESH_TOKEN_KEY, authData.refreshToken);
        await AsyncStorage.setItem(this.USER_KEY, JSON.stringify(authData.user));

        // Store session expiry
        const expiryTime = Date.now() + (authData.expiresIn * 1000);
        await AsyncStorage.setItem('session_expiry', expiryTime.toString());

    } catch (error) {
        console.error('Failed to store auth data:', error);
        throw error;
    }
}

/**
 * Clear all authentication data
 */
private async clearAuthData(): Promise<void> {
    try {
        // Clear AsyncStorage
        await AsyncStorage.multiRemove([
            this.TOKEN_KEY,
            this.REFRESH_TOKEN_KEY,
            this.USER_KEY,
            'session_expiry',
        ]);

        // Clear Keychain

```

```

    await Keychain.resetInternetCredentials('app_auth');

} catch (error) {
    console.error('Failed to clear auth data:', error);
}
}

/**
 * Set up automatic token refresh
 */
private async setupTokenRefresh(): Promise<void> {
    const expiryTime = await AsyncStorage.getItem('session_expiry');

    if (expiryTime) {
        const timeUntilExpiry = parseInt(expiryTime) - Date.now();
        const refreshTime = Math.max(timeUntilExpiry - (5 * 60 * 1000), 60000); // 5
minutes before expiry

        this.tokenRefreshTimer = setTimeout(async () => {
            const newToken = await this.refreshToken();
            if (newToken) {
                // Set up next refresh
                await this.setupTokenRefresh();
            }
        }, refreshTime);
    }
}

/**
 * Monitor session and warn user before expiry
 */
private setupSessionMonitoring(): void {
    // Check session every minute
    const checkInterval = setInterval(async () => {
        const expiryTime = await AsyncStorage.getItem('session_expiry');

        if (expiryTime) {
            const timeUntilExpiry = parseInt(expiryTime) - Date.now();

            // Warn user 10 minutes before expiry
            if (timeUntilExpiry > 0 && timeUntilExpiry < (10 * 60 * 1000)) {
                this.showSessionWarning(Math.floor(timeUntilExpiry / 60000));
            }
        }
    }, 60000);

    // Store interval reference for cleanup
    this.sessionWarningTimer = checkInterval;
}

```

```
/**
 * Show session expiry warning
 */
private showSessionWarning(minutesLeft: number): void {
  // Implement session warning UI
  console.warn(`Session expires in ${minutesLeft} minutes`);
}

/**
 * Check device security
 */
private async checkDeviceSecurity(): Promise<void> {
  try {
    // Check if device is rooted/jailbroken
    const isJailbroken = await DeviceInfo.isEmulator();

    if (isJailbroken) {
      console.warn('Security risk: Device may be compromised');
      // Implement additional security measures
    }

    // Check screen lock
    const hasScreenLock = await LocalAuthentication.isEnrolledAsync();

    if (!hasScreenLock) {
      console.warn('Security recommendation: Enable screen lock');
    }
  } catch (error) {
    console.error('Device security check failed:', error);
  }
}

/**
 * Get device information for security
 */
private async getDeviceInfo(): Promise<DeviceInfo> {
  return {
    deviceId: await DeviceInfo.getUniqueId(),
    platform: DeviceInfo.getSystemName(),
    version: DeviceInfo.getSystemVersion(),
    model: await DeviceInfo.getDeviceId(),
    appVersion: DeviceInfo.getVersion(),
  };
}

/**
 * Encrypt sensitive data
```

```
*/
private encrypt(text: string): string {
  return CryptoJS.AES.encrypt(text, this.ENCRIPTION_KEY).toString();
}

/**
 * Decrypt sensitive data
 */
private decrypt(cipherText: string): string {
  const bytes = CryptoJS.AES.decrypt(cipherText, this.ENCRIPTION_KEY);
  return bytes.toString(CryptoJS.enc.Utf8);
}

/**
 * Verify token validity
 */
private async verifyToken(token: string): Promise<boolean> {
  try {
    const response = await fetch('/api/auth/verify', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
      },
    });

    return response.ok;
  } catch (error) {
    return false;
  }
}

/**
 * Check if biometrics are enabled
 */
private async isBiometricsEnabled(): Promise<boolean> {
  try {
    const credentials = await Keychain.getInternetCredentials('app_auth');
    return credentials !== false;
  } catch (error) {
    return false;
  }
}

/**
 * Get stored authentication token
 */
public async getStoredToken(): Promise<string | null> {
  return await AsyncStorage.getItem(this.TOKEN_KEY);
}
```

```
/**
 * Clear all timers
 */
private clearTimers(): void {
  if (this.tokenRefreshTimer) {
    clearTimeout(this.tokenRefreshTimer);
  }

  if (this.sessionWarningTimer) {
    clearInterval(this.sessionWarningTimer);
  }
}

// Export singleton instance
export const authService = AuthService.getInstance();

// TypeScript interfaces
interface LoginCredentials {
  email: string;
  password: string;
  rememberMe?: boolean;
}

interface AuthResult {
  success: boolean;
  user?: User;
  token?: string;
  error?: string;
}

interface AuthData {
  user: User;
  token: string;
  refreshToken: string;
  expiresIn: number;
}

interface DeviceInfo {
  deviceId: string;
  platform: string;
  version: string;
  model: string;
  appVersion: string;
}
```

Chapter 15: Push Notifications and Background Tasks

Advanced Notification System

Push notifications are essential for user engagement and real-time communication. A comprehensive notification system must handle various types of notifications, manage permissions, and provide rich user experiences with actions, images, and deep linking.

Notification Features:

- **Local Notifications:** Scheduled and triggered notifications
- **Push Notifications:** Remote notifications from server
- **Rich Notifications:** Images, actions, and custom layouts
- **Deep Linking:** Navigate to specific screens from notifications
- **Background Processing:** Handle notifications when app is closed

```
// services/NotificationService.js - Comprehensive Notification Service
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import * as BackgroundFetch from 'expo-background-fetch';
import * as TaskManager from 'expo-task-manager';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { Platform } from 'react-native';

/**
 * Background task for notification processing
 */
const BACKGROUND_NOTIFICATION_TASK = 'background-notification-task';

/**
 * Register background task
 */
TaskManager.defineTask(BACKGROUND_NOTIFICATION_TASK, async () => {
  try {
    // Process pending notifications
    const notificationService = NotificationService.getInstance();
    await notificationService.processPendingNotifications();

    return BackgroundFetch.BackgroundFetchResult.NewData;
  } catch (error) {
    console.error('Background notification task failed:', error);
    return BackgroundFetch.BackgroundFetchResult.Failed;
  }
});

/**
 * Notification Service Class
```

 Copy

```

    * Handles all notification-related operations
    */
class NotificationService {
    private static instance: NotificationService;
    private expoPushToken: string | null = null;
    private notificationListener: any;
    private responseListener: any;

    // Notification categories and actions
    private readonly categories = {
        MESSAGE: 'message',
        REMINDER: 'reminder',
        ALERT: 'alert',
        SOCIAL: 'social',
    };

    /**
     * Singleton pattern
     */
    public static getInstance(): NotificationService {
        if (!NotificationService.instance) {
            NotificationService.instance = new NotificationService();
        }
        return NotificationService.instance;
    }

    /**
     * Initialize notification service
     */
    public async initialize(): Promise<void> {
        try {
            // Set notification handler
            Notifications.setNotificationHandler({
                handleNotification: async (notification) => ({
                    shouldShowAlert: true,
                    shouldPlaySound: true,
                    shouldSetBadge: true,
                    priority: Notifications.AndroidNotificationPriority.HIGH,
                }),
            });

            // Register notification categories
            await this.registerNotificationCategories();

            // Request permissions
            await this.requestPermissions();

            // Get push token
            await this.registerForPushNotifications();

```

```
// Set up listeners
this.setupNotificationListeners();

// Register background task
await this.registerBackgroundTask();

console.log('Notification service initialized successfully');
} catch (error) {
  console.error('Notification service initialization failed:', error);
}
}

/**
 * Request notification permissions
 */
private async requestPermissions(): Promise<boolean> {
  try {
    const { status: existingStatus } = await
Notifications.getPermissionsAsync();
    let finalStatus = existingStatus;

    if (existingStatus !== 'granted') {
      const { status } = await Notifications.requestPermissionsAsync({
        ios: {
          allowAlert: true,
          allowBadge: true,
          allowSound: true,
          allowAnnouncements: true,
        },
      });
      finalStatus = status;
    }

    if (finalStatus !== 'granted') {
      console.warn('Push notification permission denied');
      return false;
    }

    // For Android, set up notification channel
    if (Platform.OS === 'android') {
      await this.createNotificationChannels();
    }

    return true;
  } catch (error) {
    console.error('Permission request failed:', error);
    return false;
  }
}
```

```
}

/**
 * Create Android notification channels
 */
private async createNotificationChannels(): Promise<void> {
    await Notifications.setNotificationChannelAsync('default', {
        name: 'Default',
        importance: Notifications.AndroidImportance.HIGH,
        vibrationPattern: [0, 250, 250, 250],
        lightColor: '#FF231F7C',
        sound: 'default',
    });

    await Notifications.setNotificationChannelAsync('messages', {
        name: 'Messages',
        importance: Notifications.AndroidImportance.HIGH,
        vibrationPattern: [0, 250, 250, 250],
        lightColor: '#FF231F7C',
        sound: 'message_sound',
        description: 'New message notifications',
    });

    await Notifications.setNotificationChannelAsync('reminders', {
        name: 'Reminders',
        importance: Notifications.AndroidImportance.DEFAULT,
        vibrationPattern: [0, 250],
        lightColor: '#FF231F7C',
        sound: 'reminder_sound',
        description: 'Reminder notifications',
    });
}

/**
 * Register notification categories with actions
 */
private async registerNotificationCategories(): Promise<void> {
    await Notifications.setNotificationCategoryAsync(this.categories.MESSAGE, [
        {
            identifier: 'reply',
            buttonTitle: 'Reply',
            options: {
                isDestructive: false,
                isAuthenticationRequired: false,
            },
            textInput: {
                submitButtonTitle: 'Send',
                placeholder: 'Type your reply...',
            },
        },
    ],
```

```

    },
    {
      identifier: 'mark_read',
      buttonTitle: 'Mark as Read',
      options: {
        isDestructive: false,
        isAuthenticationRequired: false,
      },
    },
  ],
]);

await Notifications.setNotificationCategoryAsync(this.categories.REMINDER, [
  {
    identifier: 'complete',
    buttonTitle: 'Complete',
    options: {
      isDestructive: false,
      isAuthenticationRequired: false,
    },
  },
  {
    identifier: 'snooze',
    buttonTitle: 'Snooze',
    options: {
      isDestructive: false,
      isAuthenticationRequired: false,
    },
  },
]);
}

/**
 * Register for push notifications and get token
 */
private async registerForPushNotifications(): Promise<string | null> {
  try {
    if (!Device.isDevice) {
      console.warn('Push notifications only work on physical devices');
      return null;
    }

    const token = await Notifications.getExpoPushTokenAsync({
      projectId: 'your-expo-project-id',
    });

    this.expoPushToken = token.data;

    // Store token locally
    await AsyncStorage.setItem('@push_token', token.data);
  }
}

```

```

    // Send token to server
    await this.sendTokenToServer(token.data);

    console.log('Push token registered:', token.data);
    return token.data;
  } catch (error) {
    console.error('Failed to register for push notifications:', error);
    return null;
  }
}

/**
 * Send push token to server
 */
private async sendTokenToServer(token: string): Promise<void> {
  try {
    const userId = await AsyncStorage.getItem('@user_id');
    const deviceId = await AsyncStorage.getItem('@device_id');

    await fetch('/api/notifications/register', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${await AsyncStorage.getItem('@auth_token')}`,
      },
      body: JSON.stringify({
        token,
        userId,
        deviceId,
        platform: Platform.OS,
      }),
    });
  } catch (error) {
    console.error('Failed to send token to server:', error);
  }
}

/**
 * Set up notification listeners
 */
private setupNotificationListeners(): void {
  // Notification received listener
  this.notificationListener = Notifications.addNotificationReceivedListener(
    (notification) => {
      console.log('Notification received:', notification);
      this.handleNotificationReceived(notification);
    }
  );
}

```

```

// Notification response listener
this.responseListener = Notifications.addNotificationResponseReceivedListener(
  (response) => {
    console.log('Notification response:', response);
    this.handleNotificationResponse(response);
  }
);
}

/**
 * Handle received notification
 */
private async handleNotificationReceived(
  notification: Notifications.Notification
): Promise<void> {
  try {
    const { data } = notification.request.content;

    // Update badge count
    await this.updateBadgeCount();

    // Store notification for later retrieval
    await this.storeNotification(notification);

    // Trigger custom handlers based on notification type
    if (data?.type === 'message') {
      await this.handleMessageNotification(notification);
    } else if (data?.type === 'reminder') {
      await this.handleReminderNotification(notification);
    }

  } catch (error) {
    console.error('Error handling received notification:', error);
  }
}

/**
 * Handle notification response (user interaction)
 */
private async handleNotificationResponse(
  response: Notifications.NotificationResponse
): Promise<void> {
  try {
    const { notification, actionIdentifier, userText } = response;
    const { data } = notification.request.content;

    // Handle different action types
    switch (actionIdentifier) {

```

```

    case 'reply':
      if (userText && data?.conversationId) {
        await this.sendQuickReply(data.conversationId, userText);
      }
      break;

    case 'mark_read':
      if (data?.messageId) {
        await this.markMessageAsRead(data.messageId);
      }
      break;

    case 'complete':
      if (data?.taskId) {
        await this.markTaskComplete(data.taskId);
      }
      break;

    case 'snooze':
      if (data?.reminderId) {
        await this.snoozeReminder(data.reminderId);
      }
      break;

    default:
      // Handle default tap (navigate to specific screen)
      this.handleDeepLink(data);
      break;
  }
} catch (error) {
  console.error('Error handling notification response:', error);
}
}

/**
 * Schedule local notification
 */
public async scheduleLocalNotification(
  title: string,
  body: string,
  data: any = {},
  trigger?: Notifications.NotificationTriggerInput
): Promise<string> {
  try {
    const identifier = await Notifications.scheduleNotificationAsync({
      content: {
        title,
        body,
        data,

```

```

        categoryIdentifier: data.category || 'default',
        sound: data.sound || 'default',
        badge: await this.getBadgeCount() + 1,
        attachments: data.attachments || [],
    },
    trigger: trigger || null, // null means immediate
});

console.log('Local notification scheduled:', identifier);
return identifier;
} catch (error) {
    console.error('Failed to schedule notification:', error);
    throw error;
}
}

/**
 * Schedule recurring notification
 */
public async scheduleRecurringNotification(
    title: string,
    body: string,
    hour: number,
    minute: number,
    data: any = {}
): Promise<string> {
    const trigger = {
        type: 'daily' as const,
        hour,
        minute,
    };

    return await this.scheduleLocalNotification(title, body, data, trigger);
}

/**
 * Cancel notification
 */
public async cancelNotification(identifier: string): Promise<void> {
    await Notifications.cancelScheduledNotificationAsync(identifier);
}

/**
 * Cancel all notifications
 */
public async cancelAllNotifications(): Promise<void> {
    await Notifications.cancelAllScheduledNotificationsAsync();
}

```

```

/**
 * Get badge count
 */
private async getBadgeCount(): Promise<number> {
  const badgeCount = await AsyncStorage.getItem('@badge_count');
  return badgeCount ? parseInt(badgeCount, 10) : 0;
}

/**
 * Update badge count
 */
private async updateBadgeCount(): Promise<void> {
  const currentCount = await this.getBadgeCount();
  const newCount = currentCount + 1;

  await AsyncStorage.setItem('@badge_count', newCount.toString());
  await Notifications.setBadgeCountAsync(newCount);
}

/**
 * Clear badge count
 */
public async clearBadgeCount(): Promise<void> {
  await AsyncStorage.setItem('@badge_count', '0');
  await Notifications.setBadgeCountAsync(0);
}

/**
 * Store notification for history
 */
private async storeNotification(
  notification: Notifications.Notification
): Promise<void> {
  try {
    const storedNotifications = await AsyncStorage.getItem('@notifications');
    const notifications = storedNotifications ? JSON.parse(storedNotifications)
: [];

    notifications.unshift({
      id: notification.request.identifier,
      title: notification.request.content.title,
      body: notification.request.content.body,
      data: notification.request.content.data,
      receivedAt: Date.now(),
      isRead: false,
    });

    // Keep only last 100 notifications
    const trimmedNotifications = notifications.slice(0, 100);

```

```

        await AsyncStorage.setItem('@notifications',
JSON.stringify(trimmedNotifications));
    } catch (error) {
        console.error('Failed to store notification:', error);
    }
}

/**
 * Get stored notifications
 */
public async getStoredNotifications(): Promise<any[]> {
    try {
        const storedNotifications = await AsyncStorage.getItem('@notifications');
        return storedNotifications ? JSON.parse(storedNotifications) : [];
    } catch (error) {
        console.error('Failed to get stored notifications:', error);
        return [];
    }
}

/**
 * Register background task for notification processing
 */
private async registerBackgroundTask(): Promise<void> {
    try {
        await BackgroundFetch.registerTaskAsync(BACKGROUND_NOTIFICATION_TASK, {
            minimumInterval: 15 * 60, // 15 minutes
            stopOnTerminate: false,
            startOnBoot: true,
        });

        console.log('Background notification task registered');
    } catch (error) {
        console.error('Failed to register background task:', error);
    }
}

/**
 * Process pending notifications in background
 */
public async processPendingNotifications(): Promise<void> {
    try {
        // Fetch pending notifications from server
        const response = await fetch('/api/notifications/pending', {
            headers: {
                'Authorization': `Bearer ${await AsyncStorage.getItem('@auth_token')}`,
            },
        });
    });
}

```

```

    if (response.ok) {
      const pendingNotifications = await response.json();

      for (const notification of pendingNotifications) {
        await this.scheduleLocalNotification(
          notification.title,
          notification.body,
          notification.data
        );
      }
    }
  } catch (error) {
    console.error('Failed to process pending notifications:', error);
  }
}

/**
 * Handle deep linking from notifications
 */
private handleDeepLink(data: any): void {
  if (data?.screen) {
    // Use your navigation service to navigate
    // NavigationService.navigate(data.screen, data.params);
    console.log('Navigate to:', data.screen, data.params);
  }
}

/**
 * Handle message-specific notifications
 */
private async handleMessageNotification(
  notification: Notifications.Notification
): Promise<void> {
  // Custom logic for message notifications
  const { data } = notification.request.content;

  // Update conversation state
  // Trigger real-time updates
  // etc.
}

/**
 * Send quick reply
 */
private async sendQuickReply(conversationId: string, message: string):
Promise<void> {
  try {
    await fetch('/api/messages/quick-reply', {

```

```

    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${await AsyncStorage.getItem('@auth_token')}`,
    },
    body: JSON.stringify({
      conversationId,
      message,
    }),
  });
} catch (error) {
  console.error('Failed to send quick reply:', error);
}
}

/**
 * Mark message as read
 */
private async markMessageAsRead(messageId: string): Promise<void> {
  try {
    await fetch(`/api/messages/${messageId}/read`, {
      method: 'PUT',
      headers: {
        'Authorization': `Bearer ${await AsyncStorage.getItem('@auth_token')}`,
      },
    });
  } catch (error) {
    console.error('Failed to mark message as read:', error);
  }
}

/**
 * Cleanup listeners
 */
public cleanup(): void {
  if (this.notificationListener) {
    Notifications.removeNotificationSubscription(this.notificationListener);
  }

  if (this.responseListener) {
    Notifications.removeNotificationSubscription(this.responseListener);
  }
}

// Export singleton instance
export const notificationService = NotificationService.getInstance();

```

Chapter 16: Building and Deployment

Production-Ready Build Process

Deploying React Native applications requires careful consideration of build optimization, code signing, and store submission processes. This chapter covers the complete deployment workflow for both iOS and Android platforms.

Deployment Considerations:

- **Code Optimization:** Minification, tree shaking, and bundle splitting
- **Asset Optimization:** Image compression and lazy loading
- **Security:** Code obfuscation and sensitive data protection
- **Performance:** Bundle size optimization and startup time
- **Store Guidelines:** App Store and Play Store requirements

```
// scripts/build-optimization.js - Build Optimization Script
const fs = require('fs');
const path = require('path');
const { execSync } = require('child_process');

/**
 * Production Build Optimization Script
 * Handles code optimization, asset compression, and security measures
 */
class BuildOptimizer {
  constructor(platform) {
    this.platform = platform;
    this.rootDir = process.cwd();
    this.buildConfig = this.loadBuildConfig();
  }

  /**
   * Load build configuration
   */
  loadBuildConfig() {
    const configPath = path.join(this.rootDir, 'build-config.json');

    if (fs.existsSync(configPath)) {
      return JSON.parse(fs.readFileSync(configPath, 'utf8'));
    }

    // Default configuration
    return {
      optimization: {
        minify: true,
        obfuscate: true,
```

 Copy

```
        removeConsole: true,
        compressImages: true,
        bundleSplit: true,
    },
    security: {
        enableCodePush: false,
        enableCrashReporting: true,
        enableAnalytics: true,
    },
    performance: {
        enableHermes: true,
        enableProguard: true,
        enableR8: true,
    },
};
}

/**
 * Main build process
 */
async build() {
    console.log(`🚀 Starting ${this.platform} production build...`);

    try {
        // Pre-build checks
        await this.runPreBuildChecks();

        // Optimize assets
        await this.optimizeAssets();

        // Configure environment
        await this.configureEnvironment();

        // Run security measures
        await this.applySecurity();

        // Build the app
        await this.buildApp();

        // Post-build optimizations
        await this.postBuildOptimizations();

        // Generate build report
        await this.generateBuildReport();

        console.log('✅ Build completed successfully!');
    } catch (error) {
        console.error('❌ Build failed:', error);
    }
}
```

```

        process.exit(1);
    }
}

/**
 * Run pre-build checks
 */
async runPreBuildChecks() {
    console.log('🔍 Running pre-build checks...');

    // Check Node.js version
    const nodeVersion = process.version;
    const requiredVersion = 'v16.0.0';

    if (nodeVersion < requiredVersion) {
        throw new Error(`Node.js ${requiredVersion} or higher required. Found ${nodeVersion}`);
    }

    // Check dependencies
    console.log('📦 Checking dependencies...');
    execSync('npm audit --production', { stdio: 'inherit' });

    // Run tests
    console.log('🧪 Running tests...');
    execSync('npm test -- --passWithNoTests', { stdio: 'inherit' });

    // Lint code
    console.log('🔧 Linting code...');
    execSync('npm run lint', { stdio: 'inherit' });

    // Type check (if using TypeScript)
    if (fs.existsSync(path.join(this.rootDir, 'tsconfig.json'))) {
        console.log('📝 Type checking...');
        execSync('npx tsc --noEmit', { stdio: 'inherit' });
    }
}

/**
 * Optimize assets
 */
async optimizeAssets() {
    console.log('🖼️ Optimizing assets...');

    if (this.buildConfig.optimization.compressImages) {
        await this.compressImages();
    }

    await this.optimizeFonts();
}

```

```
    await this.generateIconSets();
  }

  /**
   * Compress images
   */
  async compressImages() {
    const imageDir = path.join(this.rootDir, 'assets/images');

    if (!fs.existsSync(imageDir)) {
      return;
    }

    try {
      // Install imagemin if not present
      execSync('npm install --no-save imagemin imagemin-mozjpeg imagemin-pngquant');

      const imagemin = require('imagemin');
      const imageminMozjpeg = require('imagemin-mozjpeg');
      const imageminPngquant = require('imagemin-pngquant');

      await imagemin([`${imageDir}/**/*.{jpg,png}`], {
        destination: path.join(imageDir, 'optimized'),
        plugins: [
          imageminMozjpeg({ quality: 80 }),
          imageminPngquant({ quality: [0.6, 0.8] }),
        ],
      });

      console.log('✅ Images compressed');
    } catch (error) {
      console.warn('⚠️ Image compression failed:', error.message);
    }
  }

  /**
   * Configure build environment
   */
  async configureEnvironment() {
    console.log('⚙️ Configuring environment...');

    // Set production environment variables
    const envConfig = {
      NODE_ENV: 'production',
      API_URL: process.env.PROD_API_URL || 'https://api.yourapp.com',
      ENABLE_FLIPPER: '0',
      ENABLE_LOGS: '0',
    };
  }
}
```

```

// Write environment file
const envContent = Object.entries(envConfig)
  .map([[key, value]] => `${key}=${value}`)
  .join('\n');

fs.writeFileSync(path.join(this.rootDir, '.env.production'), envContent);

// Configure Metro bundler
await this.configureMetro();

// Configure platform-specific settings
if (this.platform === 'ios') {
  await this.configureIOS();
} else if (this.platform === 'android') {
  await this.configureAndroid();
}
}

/**
 * Configure Metro bundler for production
 */
async configureMetro() {
  const metroConfig = `
const { getDefaultConfig } = require('metro-config');

module.exports = (async () => {
  const {
    resolver: { sourceExts, assetExts },
  } = await getDefaultConfig();

  return {
    transformer: {
      babelTransformerPath: require.resolve('react-native-svg-transformer'),
      minifierPath: require.resolve('metro-minify-terser'),
      minifierConfig: {
        mangle: true,
        output: {
          comments: false,
        },
        sourceMap: false,
      },
    },
    resolver: {
      assetExts: assetExts.filter(ext => ext !== 'svg'),
      sourceExts: [...sourceExts, 'svg'],
    },
    serializer: {
      customSerializer: require('metro-serializer-esbuild')({

```

```

        minify: true,
        target: 'es2017',
      }},
    },
  };
})();
`;

fs.writeFileSync(path.join(this.rootDir, 'metro.config.js'), metroConfig);
}

/**
 * Configure iOS-specific settings
 */
async configureIOS() {
  console.log('🍏 Configuring iOS build...');

  // Enable Hermes
  if (this.buildConfig.performance.enableHermes) {
    const podfilePath = path.join(this.rootDir, 'ios/Podfile');

    if (fs.existsSync(podfilePath)) {
      let podfileContent = fs.readFileSync(podfilePath, 'utf8');

      if (!podfileContent.includes(':hermes_enabled => true')) {
        podfileContent = podfileContent.replace(
          ':hermes_enabled => false',
          ':hermes_enabled => true'
        );

        fs.writeFileSync(podfilePath, podfileContent);
        console.log('✅ Hermes enabled for iOS');
      }
    }
  }

  // Configure build settings
  const buildSettingsPath = path.join(this.rootDir, 'ios/YourApp/Info.plist');

  if (fs.existsSync(buildSettingsPath)) {
    // Update Info.plist for production
    execSync(`plutil -replace CFBundleDisplayName -string "Your App"
"${buildSettingsPath}"`);
    execSync(`plutil -replace CFBundleVersion -string "${this.getBuildNumber()}"
"${buildSettingsPath}"`);
  }
}

/**

```

```
* Configure Android-specific settings
*/
async configureAndroid() {
  console.log('🤖 Configuring Android build...');

  // Enable Hermes
  if (this.buildConfig.performance.enableHermes) {
    const gradlePath = path.join(this.rootDir, 'android/app/build.gradle');

    if (fs.existsSync(gradlePath)) {
      let gradleContent = fs.readFileSync(gradlePath, 'utf8');

      if (!gradleContent.includes('enableHermes: true')) {
        gradleContent = gradleContent.replace(
          'enableHermes: false',
          'enableHermes: true'
        );

        fs.writeFileSync(gradlePath, gradleContent);
        console.log('✅ Hermes enabled for Android');
      }
    }
  }

  // Enable R8/ProGuard
  if (this.buildConfig.performance.enableR8) {
    const gradlePath = path.join(this.rootDir, 'android/app/build.gradle');

    if (fs.existsSync(gradlePath)) {
      let gradleContent = fs.readFileSync(gradlePath, 'utf8');

      gradleContent = gradleContent.replace(
        'minifyEnabled false',
        'minifyEnabled true'
      );

      gradleContent = gradleContent.replace(
        'shrinkResources false',
        'shrinkResources true'
      );

      fs.writeFileSync(gradlePath, gradleContent);
      console.log('✅ R8/ProGuard enabled');
    }
  }

  // Update version code and name
  const versionCode = this.getBuildNumber();
  const versionName = this.getVersionName();
```

```

    execSync(`cd android && ./gradlew -q app:setVersionCode -
PversionCode=${versionCode}`);
    execSync(`cd android && ./gradlew -q app:setVersionName -
PversionName=${versionName}`);
  }

/**
 * Apply security measures
 */
async applySecurity() {
  console.log('🔒 Applying security measures...');

  if (this.buildConfig.optimization.removeConsole) {
    await this.removeConsoleLogs();
  }

  if (this.buildConfig.optimization.obfuscate) {
    await this.obfuscateCode();
  }

  await this.validateSecrets();
}

/**
 * Remove console logs from production build
 */
async removeConsoleLogs() {
  const babelConfigPath = path.join(this.rootDir, 'babel.config.js');

  if (fs.existsSync(babelConfigPath)) {
    let babelConfig = fs.readFileSync(babelConfigPath, 'utf8');

    // Add transform plugin to remove console statements
    if (!babelConfig.includes('transform-remove-console')) {
      babelConfig = babelConfig.replace(
        'plugins: [' ,
        `plugins: [
          ['transform-remove-console', { exclude: ['error', 'warn'] }],`
      );
    }

    fs.writeFileSync(babelConfigPath, babelConfig);
  }
}

/**
 * Validate that no secrets are exposed
 */

```

```

async validateSecrets() {
  const filesToCheck = [
    'src/**/*.js',
    'src/**/*.ts',
    'src/**/*.tsx',
  ];

  const secretPatterns = [
    /api[_-]?key/i,
    /secret[_-]?key/i,
    /private[_-]?key/i,
    /password/i,
    /token.*=.*['"]/i,
  ];

  try {
    const grep = execSync(
      `grep -r -i -E "${secretPatterns.map(p => p.source).join('|')}"
${filesToCheck.join(' ')}`,
      { encoding: 'utf8' }
    );

    if (grep.trim()) {
      console.warn('⚠️ Potential secrets found in code:');
      console.warn(grep);

      if (process.env.CI) {
        throw new Error('Secrets detected in code. Build failed.');

```

```
}

/**
 * Build iOS application
 */
async buildIOS() {
  // Clean build folder
  execSync('cd ios && xcodebuild clean', { stdio: 'inherit' });

  // Install pods
  execSync('cd ios && pod install', { stdio: 'inherit' });

  // Build for release
  const buildCommand = `
    cd ios && xcodebuild archive
    -workspace YourApp.xcworkspace
    -scheme YourApp
    -configuration Release
    -destination generic/platform=iOS
    -archivePath build/YourApp.xcarchive
    CODE_SIGN_STYLE=Manual
    PROVISIONING_PROFILE_SPECIFIER="YourApp Production"
    CODE_SIGN_IDENTITY="iPhone Distribution: Your Company"
  `.replace(/\s+/g, ' ').trim();

  execSync(buildCommand, { stdio: 'inherit' });

  // Export IPA
  const exportCommand = `
    cd ios && xcodebuild -exportArchive
    -archivePath build/YourApp.xcarchive
    -exportPath build/
    -exportOptionsPlist exportOptions.plist
  `.replace(/\s+/g, ' ').trim();

  execSync(exportCommand, { stdio: 'inherit' });

  console.log('✅ iOS build completed');
}

/**
 * Build Android application
 */
async buildAndroid() {
  // Clean build
  execSync('cd android && ./gradlew clean', { stdio: 'inherit' });

  // Build release APK/AAB
  const buildCommand = this.buildConfig.buildType === 'aab'
```

```

? 'cd android && ./gradlew bundleRelease'
: 'cd android && ./gradlew assembleRelease';

execSync(buildCommand, { stdio: 'inherit' });

console.log('✅ Android build completed');
}

/**
 * Post-build optimizations
 */
async postBuildOptimizations() {
  console.log('⚡ Running post-build optimizations...');

  // Analyze bundle size
  await this.analyzeBundleSize();

  // Generate source maps for crash reporting
  if (this.buildConfig.security.enableCrashReporting) {
    await this.generateSourceMaps();
  }
}

/**
 * Analyze bundle size
 */
async analyzeBundleSize() {
  try {
    if (this.platform === 'ios') {
      const ipaPath = path.join(this.rootDir, 'ios/build/YourApp.ipa');
      if (fs.existsSync(ipaPath)) {
        const stats = fs.statSync(ipaPath);
        console.log(`📦 iOS bundle size: ${((stats.size / 1024 /
1024).toFixed(2))} MB`);
      }
    } else if (this.platform === 'android') {
      const apkPath = path.join(this.rootDir,
'android/app/build/outputs/apk/release/app-release.apk');
      const aabPath = path.join(this.rootDir,
'android/app/build/outputs/bundle/release/app-release.aab');

      if (fs.existsSync(apkPath)) {
        const stats = fs.statSync(apkPath);
        console.log(`📦 Android APK size: ${((stats.size / 1024 /
1024).toFixed(2))} MB`);
      }

      if (fs.existsSync(aabPath)) {
        const stats = fs.statSync(aabPath);

```

```

        console.log(`📦 Android AAB size: ${stats.size / 1024 /
1024}).toFixed(2)} MB`);
    }
}
} catch (error) {
    console.warn('⚠️ Bundle size analysis failed:', error.message);
}
}

/**
 * Generate build report
 */
async generateBuildReport() {
    const report = {
        platform: this.platform,
        buildDate: new Date().toISOString(),
        version: this.getVersionName(),
        buildNumber: this.getBuildNumber(),
        configuration: this.buildConfig,
        gitCommit: this.getGitCommit(),
        nodeVersion: process.version,
    };

    const reportPath = path.join(this.rootDir, `build-
report-${this.platform}.json`);
    fs.writeFileSync(reportPath, JSON.stringify(report, null, 2));

    console.log(`📊 Build report generated: ${reportPath}`);
}

/**
 * Helper methods
 */
getBuildNumber() {
    return process.env.BUILD_NUMBER || Math.floor(Date.now() / 1000).toString();
}

getVersionName() {
    const packageJson = JSON.parse(fs.readFileSync(path.join(this.rootDir,
'package.json'), 'utf8'));
    return packageJson.version;
}

getGitCommit() {
    try {
        return execSync('git rev-parse HEAD', { encoding: 'utf8' }).trim();
    } catch {
        return 'unknown';
    }
}

```

```

    }
  }

  // CLI interface
  if (require.main === module) {
    const platform = process.argv[2];

    if (!platform || ![ 'ios', 'android' ].includes(platform)) {
      console.error('Usage: node build-optimization.js <ios|android>');
      process.exit(1);
    }

    const optimizer = new BuildOptimizer(platform);
    optimizer.build().catch(error => {
      console.error('Build failed:', error);
      process.exit(1);
    });
  }

  module.exports = BuildOptimizer;

```

Continuous Integration/Deployment Pipeline

.github/workflows/ci-cd.yml - GitHub Actions CI/CD Pipeline

 Copy

name: React Native CI/CD

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

env:

NODE_VERSION: '16.x'

JAVA_VERSION: '11'

jobs:

Quality checks

lint-and-test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Setup Node.js

uses: actions/setup-node@v3

with:

node-version: \${ env.NODE_VERSION }

```
    cache: 'npm'

- name: Install dependencies
  run: npm ci

- name: Run ESLint
  run: npm run lint

- name: Run TypeScript check
  run: npx tsc --noEmit

- name: Run tests
  run: npm test -- --coverage --watchAll=false

- name: Upload coverage
  uses: codecov/codecov-action@v3

# Security checks
security-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3

    - name: Run security audit
      run: npm audit --audit-level high

    - name: Scan for secrets
      uses: trufflesecurity/trufflehog@main
      with:
        path: ./
        base: main

# Android build and deployment
build-android:
  runs-on: ubuntu-latest
  needs: [lint-and-test, security-scan]
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Setup JDK
      uses: actions/setup-java@v3
```

```

with:
  java-version: ${ env.JAVA_VERSION }
  distribution: 'temurin'

- name: Setup Android SDK
  uses: android-actions/setup-android@v2

- name: Install dependencies
  run: npm ci

- name: Cache Gradle
  uses: actions/cache@v3
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
    key: gradle-${ runner.os }-${ hashFiles('**/*.gradle*', '**/gradle-
wrapper.properties') }}

- name: Decode Android keystore
  run: |
    echo "${ secrets.ANDROID_KEYSTORE_BASE64 }}" | base64 -d >
    android/app/release.keystore

- name: Create key.properties
  run: |
    echo "storeFile=release.keystore" > android/key.properties
    echo "storePassword=${ secrets.ANDROID_STORE_PASSWORD }}" >>
    android/key.properties
    echo "keyAlias=${ secrets.ANDROID_KEY_ALIAS }}" >>
    android/key.properties
    echo "keyPassword=${ secrets.ANDROID_KEY_PASSWORD }}" >>
    android/key.properties

- name: Build Android AAB
  run: |
    cd android
    ./gradlew bundleRelease

- name: Upload AAB to Play Console
  uses: r0adk11/upload-google-play@v1
  with:
    serviceAccountJsonPlainText: ${ secrets.GOOGLE_PLAY_SERVICE_ACCOUNT }
    packageName: com.yourapp.android
    releaseFiles: android/app/build/outputs/bundle/release/app-release.aab
    track: internal
    status: draft

```

iOS build and deployment

```
build-ios:
  runs-on: macos-latest
  needs: [lint-and-test, security-scan]
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v3

    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Setup Ruby
      uses: ruby/setup-ruby@v1
      with:
        ruby-version: '3.0'
        bundler-cache: true
        working-directory: ios

    - name: Install CocoaPods
      run: |
        cd ios
        pod install

    - name: Import certificates
      uses: Apple-Actions/import-codesign-certs@v1
      with:
        p12-file-base64: ${ secrets.IOS_P12_BASE64 }
        p12-password: ${ secrets.IOS_P12_PASSWORD }

    - name: Import provisioning profile
      uses: Apple-Actions/download-provisioning-profiles@v1
      with:
        bundle-id: com.yourapp.ios
        profile-type: IOS_APP_STORE
        issuer-id: ${ secrets.APPSTORE_ISSUER_ID }
        api-key-id: ${ secrets.APPSTORE_KEY_ID }
        api-private-key: ${ secrets.APPSTORE_PRIVATE_KEY }

    - name: Build and archive
      run: |
        cd ios
        xcodebuild archive \
          -workspace YourApp.xcworkspace \
```

```

    -scheme YourApp \
    -configuration Release \
    -destination generic/platform=iOS \
    -archivePath build/YourApp.xcarchive \
    CODE_SIGN_STYLE=Manual \
    PROVISIONING_PROFILE_SPECIFIER="YourApp Production" \
    CODE_SIGN_IDENTITY="iPhone Distribution"

- name: Export IPA
  run: |
    cd ios
    xcodebuild -exportArchive \
      -archivePath build/YourApp.xcarchive \
      -exportPath build/ \
      -exportOptionsPlist exportOptions.plist

- name: Upload to TestFlight
  uses: Apple-Actions/upload-testflight-build@v1
  with:
    app-path: ios/build/YourApp.ipa
    issuer-id: ${ secrets.APPSTORE_ISSUER_ID }
    api-key-id: ${ secrets.APPSTORE_KEY_ID }
    api-private-key: ${ secrets.APPSTORE_PRIVATE_KEY }

# Deployment notification
notify-deployment:
  runs-on: ubuntu-latest
  needs: [build-android, build-ios]
  if: always()

  steps:
    - name: Notify team
      uses: 8398a7/action-slack@v3
      with:
        status: ${ job.status }
        channel: '#deployments'
        webhook_url: ${ secrets.SLACK_WEBHOOK }
        message: |
          React Native app deployment completed
          Android: ${ needs.build-android.result }
          iOS: ${ needs.build-ios.result }

```

Course Completion Summary

Congratulations! 🎉 You've completed the comprehensive React Native course from scratch to expert level. This course covered:

Part 1 - Foundations:

- Environment setup and project structure
- Core components and styling
- Layout with Flexbox
- User interactions and forms

Part 2 - Core Development:

- Navigation patterns and screen management
- State management techniques
- List rendering and data handling
- API integration and networking

Part 3 - Advanced Features:

- Native device features (Camera, Location, Storage)
- Animations and advanced UI patterns
- Performance optimization strategies
- Comprehensive testing and debugging

Part 4 - Professional Development:

- Enterprise-level state management with Redux
- Security and authentication systems
- Push notifications and background tasks
- Production build and deployment processes

Next Steps for Your React Native Journey:

1. **Build Portfolio Projects:** Create 3-5 diverse apps showcasing different skills
2. **Contribute to Open Source:** Join React Native community projects
3. **Stay Updated:** Follow React Native releases and new features
4. **Learn Platform-Specific Features:** Dive deeper into iOS and Android native development
5. **Explore Advanced Topics:** Web3 integration, AR/VR, IoT connectivity

Professional Development Resources:

- **React Native Documentation:** Official guides and API references
- **GitHub Repositories:** Study popular React Native apps